

A Decentralized Exchange on the Cost-Accounted Rho Calculus

A gap analysis, a two-spine decomposition, and a build plan for a hybrid
spot-and-derivatives venue on F1R3Fi

F1R3FLY.io

L. G. Meredith *in dialogue with Anthropic's Claude*

July 2026 — Draft for discussion

Abstract

The Peyton Jones–Eber–Seward combinators, realised in the cost-accounted rho calculus, already give an executable, resource-safe settlement language; two venues have been built on top of it — Priced Plausible Fiction (a two-sided market of programmatic intentions) and a Kalshi/Polymarket-class prediction market. This note asks what remains to stand up a third and more demanding application on the same substrate: a full decentralised exchange (DEX), hybrid in market structure (automated market maker *and* order book), permissioned or permissionless by configuration, and spanning spot trading, derivatives, and lending. The organising result is a *two-spine decomposition*. The *spot spine* settles entirely on-substrate and is oracle-free: it inherits atomic settlement, located authority, and MEV-flat batch clearing from the existing apparatus, and its swap primitive is not assembled but is a *theorem* of the join schema (conservation of authority). The *derivatives spine* re-imports the prediction-market oracle discipline wholesale and forces exactly one genuinely new risk-centre object, the liquidation engine, which we show is the acceptance gate lifted from a one-shot deployment check to a standing margin invariant — and, operationally, a health-triggered variant of the loan-sale swap already implemented. We identify the multi-asset ledger as a free consequence of the cost *spectrum* (each asset a signature-indexed located line), fix the one design decision this leaves open (value as magnitude-in-purse, metered by the unary phlogiston line, kept apart by *cost is not amount*), and argue that the tensor–hom adjunction on signatures ($*$ = $\otimes$, $\multimap$ the internal hom) is the intuitionistic-linear-logic backbone of the venue: an asset is a linear resource, a swap a linear map, a multi-hop route a curried composition, and a well-typed atomic route a single linear proof. We give a DEX-launch configuration tuple over a fixed safety floor, a primitive-by-primitive map against four papers and five implementation files, an honest ledger of what is present versus new, and a phased build plan. The headline: the spot swap, baskets, secondary-market position sale, MEV-resistant clearing, and term-lending *instrument* are present or nearly so; the genuinely new builds are the stateful pool object, the liquidation engine, cross-pool batch clearing, and the money-market rate model — and the credit *venue*, as distinct from the credit instrument, does not yet exist.

Contents

1	Introduction	2
2	Background: the apparatus we build on	3
3	The two-spine decomposition	4
4	The asset layer: the cost spectrum <i>is</i> the multi-asset ledger	5

5	The spot pool: the core new object	6
6	Clearing and MEV: batch is idiomatic, cross-pool is the hard problem	7
7	The derivatives spine: the oracle returns, mostly reused	7
8	The liquidation engine: the acceptance gate made standing	8
9	Lending: the instrument is present, the venue is not	8
10	Structural findings	9
10.1	The swap is a theorem, not a construction	9
10.2	The ILL backbone: asset as linear resource, route as curried composition	10
10.3	Flash swaps: an elegant fit, not a friction	10
11	The DEX-launch configuration tuple over a fixed safety floor	10
12	Primitive-by-primitive map	12
13	Phased build plan	12
14	Established vs. conjectural; open problems	13
15	Conclusion	14

1 Introduction

A financial contract, in the Peyton Jones–Eber–Seward (PJES) sense [10], is a thing one *holds*. A prediction market [7] is a *venue* one trades in. A job market of programmatic intentions [6] is a venue of a different shape. A decentralised exchange is a fourth object again — and, in the modern DeFi sense of the term, the most demanding, because “DEX” has come to mean not one venue but a stack of them: a spot exchange (automated market maker and/or order book), a derivatives exchange (perpetuals, options), and a money market (lending and borrowing), sharing a collateral and settlement layer.

The thesis of this note is that the F1R3Fi substrate — the cost-accounted rho calculus [3] obtained by applying the cost endofunctor [4] to the reflective higher-order rho calculus [1], with the PJES combinators realised on top [5] — is unusually well-suited to carrying this stack, that a surprising fraction of it is *already present* in the four papers and five implementation files we build on, and that the residue concentrates into four new objects rather than a new platform. We make the case by inventory and construction, in the spirit of the prediction-market gap analysis [7], which decomposed a venue into four layers and located each against existing apparatus.

The organising insight: two spines. The single most useful thing to say at the outset is that a *spot* DEX and a *derivatives* DEX differ in exactly one structural respect, and it is the respect that decides where the difficulty lives.

- The **spot spine** (asset-for-asset swaps, AMM pools, order books, routing) settles *immediately and entirely on-substrate*. There is no future event to recognise, no attestation, no off-chain dispatch, no resolution latch, no dispute window. This spine *escapes* the oracle trust model

that both prior venues live inside [6, §8.5]. The riskiest and least substrate-native layer of the prediction market — Layer 3, oracle/resolution, where the venue “lives or dies” [7, §3] — simply does not arise for spot.

- The **derivatives spine** (perpetuals, options, lending) needs external price for mark, settlement, and collateral valuation, so the oracle discipline of [7, §4] is *reused wholesale*, not rebuilt. Option settlement *is* the prediction-market resolution latch. What this spine forces that is genuinely new is a liquidation engine — its risk centre, exactly as the oracle was the prediction market’s.

Schematically, the full venue is

$$\text{DEX} \approx \underbrace{\text{asset ledger}}_{\text{cost spectrum, §4}} \cup \underbrace{\text{spot pool + clearing}}_{\text{new: §5, §6}} \cup \underbrace{\text{pred.-market oracle apparatus}}_{\text{reused: [7]}} \cup \underbrace{\text{liquidation engine}}_{\text{new: §8}}.$$

What we take as given. We take as given: the reflective rho calculus [1] and its namespace logic generator OSLF [2]; the cost-accounted calculus [3] with located token stacks, the acceptance gate, signatures-as-names, the five rewrite rules, the join cost schema, and the tensor-hom remark; the PJES realisation [5] with its four cost-accounting patterns and the located-stack correction; the priced-market machinery [6]; the prediction-market venue apparatus [7]; and the five `spj_*cost.rho` implementation files, of whose shared cost kernel and loan/futures layers we have read the source directly.¹

Outline. §2 recalls the load-bearing apparatus. §3 develops the two-spine decomposition. §4 shows the multi-asset ledger falling out of the cost spectrum and fixes the value-representation decision. §§5–6 treat the two genuinely new spot builds: the stateful pool and cross-pool clearing. §7 treats the derivatives spine and §8 the liquidation engine. §9 audits what lending is present and what a credit *venue* still needs. §10 collects the structural findings (swap-as-join, the ILL backbone, flash-as- $\rightarrow$). §11 gives the launch configuration tuple over a safety floor. §12 is the primitive-by-primitive map; §13 the phased plan; §14 the honest ledger of established versus conjectural. §15 concludes.

2 Background: the apparatus we build on

We recall only what is load-bearing; [1, 3, 4, 5] give the full account.

The reflective calculus. Names are quoted processes; the grammar is $P, Q ::= 0 \mid \text{for}(y \leftarrow x) P \mid x!(Q) \mid P \mid Q \mid *x$, with $x, y ::= @P$. Reflection lets contracts, observables, oracles and obligations all be processes, exchanged as names and run by dereference. Parallel composition is associative-commutative (AC): the process soup is an unordered bag, which is what forces the located-stack discipline.

¹The cost kernel (`mkSlot`, `reserve`, `reserve2`, `spend`, `refund`, `acquire`) and the combinators (`makeOne`, `makeGive`, `makeCand`, `makeTruncate`, `makeGet`) together with the loan constructors (`schedulePayment`, `makeScheduledPayments`, `makeShortTermLoan`, `makeMidTermLoan`, `makeLongTermLoan`, `makeSellLoan`) are cited from the verified source of `spj_loans.cost.rho` and the header/kernel of `spj-futures-cost.rho`. The full combinator family and the auction/power instruments are cited at the level documented in [5, §2.2, §4], attributed to `spj_contracts.cost.rho`, `spj_dutch_auction.cost.rho`, and `spj-power-contracts-cost.rho`.

Cost accounting. The cost endofunctor [4] adjoins signatures as a species of name $(x, y ::= @T \mid s)$, token stacks as first-class terms $(S ::= () \mid s :: S, \text{ a temporal monoid})$, and wrapping-by-construction (every continuation a signed thunk $\{P\}_s$ that fires only by spending a matching token). The single communication rule becomes a gated family whose base case is $\{\text{for}(y \leftarrow x)T \mid x!(U)\}_s \mid s :: S \rightsquigarrow T\{\text{@}U/y\} \mid S$. Three facts we use constantly:

- (i) **Located purses** [5, §3.2]. A stack held on a channel $c!(s :: S)$; the right to spend is the right to receive on c . Because **new**-minted channels are unforgeable, authority is possession of the name, never adjacency in the AC bag.
- (ii) **The acceptance gate** [3, §8.5]. Multi-step financial atomicity is a static analysis at deployment-acceptance: a linear resource proof $\Sigma_s \geq \Delta_s$ (supply at least demand) for every signature, *before any step fires*. Its operational form is the all-or-nothing **reserve** from a located slot.
- (iii) **Cost is not amount** [5, §3.6]. Phlogiston counts funded rendezvous, not the magnitude of value moved; a swap of any size costs one token.

The join schema and the two monoids. A join is an N -ary cut firing all-or-nothing across N channels. The join cost schema quantifies over three partition axes (receiver clauses, sends, token presentation), and Proposition 4.7 of [3] (*conservation of authority*) states that, across every choice of partition, the multiset of signatures consumed is exactly one receiver-authority and one sender-authority per channel: “the join either gathers one token’s worth of every participant’s authority, or it does not fire.” Two monoids sit side by side: the temporal stack monoid $::$ (what is spent next, admits benign head-consumption) and the spatial-signature monoid $*$ (compound authority, *weakening-forbidden* — a composite must be discharged in full or explicitly **Split**). Remark 4.9 of [3] makes $*$ a tensor and the linear transfer $\multimap$ its right adjoint, leaving open whether the token discipline is a model of intuitionistic linear logic (ILL); §10 argues that for a DEX this open question *is* the type theory.

The four cost-accounting patterns. [5] classifies every realised instrument into four patterns: (A) bilateral cash settlement / netting; (B) single-funder fixed leg; (C) conservative-bound option (max over branches, refund the unforced); (D) time-separated bundle (each leg its own funded deployment under concurrent acceptance). These recur throughout the DEX.

The implementation kernel. The five files share one cost kernel. **reserve** is the acceptance gate’s **peel** loop verbatim; **reserve2** realises the compound-signature $s_1 * s_2$ join (Rule 2), restoring both slots if either party is unfunded; **refund** returns the unforced remainder; **acquire** is the per-deployment boundary. On this kernel sit the combinators and, in **spj_loans_cost.rho**, term lending and the loan-sale atomic swap. We lean on these directly in §§8–9.

3 The two-spine decomposition

Table 1 states the decomposition. The consequence for build risk is immediate: the part of a DEX that DeFi practitioners find hardest — atomic, non-stranding, MEV-resistant settlement — is the part this substrate has *already* solved, three times over ([5, 6, 7]). The part that is actually new (a persistent, contended, autonomous counterparty holding *independent* assets, plus a solvency-triggered forced-unwind object) is the part no prior venue had to construct, because every

	Spot spine	Derivatives spine
Settlement	On-substrate, immediate, atomic	On-substrate settlement of an oracle-priced obligation
Oracle	None (self-referential price only)	Reused wholesale from [7, §4]
Risk centre	Contended pool state (dissolved by batch clearing)	Liquidation engine (§8)
Swap primitive	Compound-signature join — a <i>theorem</i> (§10)	Mark/settle against attested observable
New objects	Stateful pool; cross-pool clearing	Liquidation; margin accounting; money-market rate model
Inherited	Located custody, gate, netting, MEV-flat batch	Option latch, blame capability, dispute window

Table 1: The two spines. The spot spine is oracle-free and on-substrate; the derivatives spine re-imports the prediction-market oracle apparatus and forces one new risk-centre object.

prior object is either bilateral (a contract fixing its parties) or bearer-conditional (outcome shares conserved to a single collateral). A pool is neither.

We now proceed layer by layer, giving each a verdict in the vocabulary of the prediction-market note: *present*, *thin wrapper*, *new*, or *reused*.

4 The asset layer: the cost spectrum *is* the multi-asset ledger

Layer A collapses. A DEX presupposes many independent fungible assets. The spectral decomposition of phlogiston [3, §5.6] supplies them for free: phlogiston is not one fuel but “as many distinct fuels as there are economically empowered actors” — a signature-indexed spectrum. Each asset is simply another spectral line, a signature-indexed located pool. Everything an asset layer needs is a consequence of the signature algebra already in the calculus:

- **Minting** is trivial [3, §2.4]: construct a stack, send it on the asset’s signature channel. No privileged runtime hook, no oracle. [3] explicitly names ERC-20-like fungible tokens as “expressible as standard cost-accounted Rholang programs.”
- **Custody and balances** are located stacks on unforgeable channels [5, §3.2]; authority is possession of the channel.
- **Transfer** is the gated communication; **attenuation**, **amplification**, **delegation**, **just-in-time funding** all fall out of the signature algebra [3, §2.4].
- **Swap** is already witnessed: the **Exchange** contract of [3, App. B.9] is a token swap, and [3] itself notes that “more sophisticated exchange contracts can implement variable rates, order books, or automated market makers — all within Rholang.”

The DEX is therefore not greenfield on top of these papers; it is the fulfilment of a direction the foundational paper explicitly names. REV is implemented (a **RevVault** the loan and futures files call through the registry), so at least one value line exists in production today.

The one decision this leaves: three kinds of spectral line. The phlogiston line is essentially *unary* — one token per funded rendezvous — which is right for metering and wrong for asset magnitudes (one does not want a million-deep stack to represent a million units). *Cost is not amount* [5, §3.6] is exactly the seam that keeps these apart, and it forces the venue to carry (at least) three kinds of line, which must never be conflated:

- (1) the **phlogiston line** Φ (unary; meters rendezvous; the acceptance gate’s currency);
- (2) the **value lines** (the traded assets $X, Y, \dots$; magnitudes held in located purses, as REV amounts already are — “nanoREV” in the implementation);
- (3) the **share lines** (LP / pool-share and debt tokens; themselves minted spectral lines).

This is the priced-market $\mathcal{Q}$ -versus- Φ separation [6, §6] generalised: value lines carry magnitude, the phlogiston line carries count, and a deployment is admitted only against the *product* order, never their conflation.

Finding 1 (Value representation). Represent an asset amount as a *magnitude in a located purse* (as REV nanoREV already is), not as stack depth. Reserve stack depth for the unary phlogiston line and for capability-like, small-count tokens. Custody, authority, atomic settlement, and metering are identical either way — all gated communication on signature-indexed located channels — so the choice is purely the representation of magnitude, and magnitude-in-purse is the only scalable one. The *asset ledger is present*; only this convention needs fixing, and it is the DEX note’s first design decision.

5 The spot pool: the core new object

Why the pool is genuinely new. The prediction-market maker (CPMM, LMSR, CLOB) [7, §6] prices *outcome shares* under a conservation-to-collateral law $\sum_i \text{shares}_i = \text{collateral}$ — itself a located-purse funding proof. A DEX pool is a different accounting object: two (or N) *independent* assets on separate located purses under an AMM curve invariant, with fungible LP claims. The pricing-rule apparatus of [7, §6.1] transfers verbatim; what is new is the two-independent-asset invariant and the LP lifecycle.

Definition 1 (Constant-function pool). A pool for the pair (X, Y) is a persistent process holding two located reserve purses L_X, L_Y and maintaining an invariant $\kappa(L_X, L_Y) \geq k$ (with $\kappa = \cdot$ for constant-product, growing with accrued fees). A swap reads the reserves, computes the output from the curve, executes a compound-signature two-leg settlement (§10) that provably does not decrease κ , and republishes the updated reserves. Reserves, output computation, and settlement all live behind the gate; a quote that cannot prove its purse funded does not fire [7, §6.4].

LP accounting. Add-liquidity mints a share-line token pro-rata to the deposit; remove-liquidity burns it and releases reserves pro-rata; swap fees accrue to κ and thus to LP claims. Minting and burning are the trivial spectrum operations of §4; the pro-rata arithmetic is ordinary Rholang. The share line is a fungible receipt (§4 kind (3)) and inherits custody and transfer for free. **Verdict: new, the core spot build** — but the curve math is inherited and only the two-independent-asset invariant and the LP lifecycle are novel.

Spot price as an exported observable. A pool’s spot price is a persistent channel returning its current value — exactly the SPJ observable of [5, §2.2], which [7, §4.1] already identifies with a persistent oracle channel. So `scale(poolPrice, c)` is well-typed today; the pool exports price to the rest of the venue at no additional cost. A time-weighted average (TWAP) is a small accumulator on top. **Verdict: spot price present; TWAP a small new accumulator.**

6 Clearing and MEV: batch is idiomatic, cross-pool is the hard problem

Concurrency is substrate-native. The concurrency spine is already in place [3, §2.3,§8.8]: trades on disjoint asset pairs draw disjoint token pools and run in parallel without merge analysis; trades contending the same pool are competing linear-proof obligations of which at most one succeeds. The model asks “is this deployment funded?” where run-to-completion asked “is the tuplespace free?” [3, Rmk. 2.1].

Batch clearing dissolves the one hard case. A single price cell contended within a block is the one genuinely hard case the substrate flags. The frequent-batch-auction discipline of [7, §6.2] dissolves it: orders in a window clear at one uniform price, the curve serving as residual counterparty, and Proposition 6.1 of [7] (batch clearing is MEV-flat within a round) makes the intra-round reordering value exactly zero. Both requested market structures — AMM and CLOB — collapse into this one clearing discipline rather than two parallel engines: resting orders and the curve clear together at p^* . This is the correct architectural spine for the requested *hybrid* venue, and it matches the real-world convergence onto solver-cleared batch auctions.

The genuinely hard problem: cross-pool clearing. The single-market uniform-price batch does not immediately generalise to a venue clearing many pairs at once where arbitrage should be *internalised* into the clear — a coincidence-of-wants ring $X \rightarrow Y \rightarrow Z \rightarrow X$ that nets to zero external flow. As a compound-signature N -leg join, such a ring settles all-or-nothing with no merge analysis (§10); but *finding* the welfare-maximising multi-asset clear is a matching/optimisation problem, the DEX cousin of the “combinatorial maker pricing is #P-hard” caveat of [7, §3]. **Verdict: batch clearing inherited; cross-pool arbitrage internalisation is new and is where the MEV story is decided.**

7 The derivatives spine: the oracle returns, mostly reused

With “everything” in scope the oracle layer that dropped out for spot returns — but as reuse, not rebuild.

- **Perpetual mark/index** is a deterministic-stratum oracle [7, §4.3]: a persistent attested observable, source auditable by the signature-as-name on the attestation.
- **Funding** is periodic Pattern-A netting [5, §4.1] driven by that observable — a single net transfer in the data-dependent direction, precisely the cash-settled futures machinery of `spj-futures-cost.rho`, whose header already derives net settlement from `cand(scale(spot,one),give(one(F)))` under `reserve2`.
- **Option settlement** is the prediction-market resolution latch [7, §7] unchanged: write-once, redeem nested below consensus finality and dispute finality. No new machinery.
- **Cross-margin** is netting-as-clearing [6, §2.3] generalised across a portfolio — a native advantage, since the substrate already collapses opposed obligations into one atomic transfer.

What is new on this spine is margin accounting (a live health observable per position) and the liquidation engine. Margin accounting is a small stateful wrapper; liquidation earns its own section. **Verdict: oracle and option-settlement reused; margin a thin wrapper; liquidation new.**

8 The liquidation engine: the acceptance gate made standing

Liquidation is the derivatives spine’s risk centre, and reading the loan-sale swap alongside the futures netting makes it markedly less greenfield than a first pass suggests.

Liquidation as a standing acceptance gate. The acceptance gate proves $\Sigma_s \geq \Delta_s$ once, at deployment. A leveraged position is deliberately under-collateralised against notional, so it looks like a permanent funding-proof failure. The resolution reframes what the gate proves: it certifies *margin*-sufficiency, not notional-sufficiency, and it is re-evaluated continuously against the mark observable.

Finding 2 (Liquidation is the gate lifted to a standing invariant). Liquidation is precisely what fires when the continuously re-evaluated funding proof would fail. It is not a mechanism bolted beside the acceptance gate; it *is* the acceptance gate lifted from a one-shot deployment-time check to a standing invariant over a position’s life. Where the deployment gate rejects an unfunded deployment before it runs, the standing gate forces settlement of a position whose margin proof has lapsed.

Liquidation as a health-triggered makeSellLoan. Operationally, the atomic seize-and-close is a swap the implementation *already contains*. The loan sale `makeSellLoan` of `spj_loans_cost.rho` sells a lender position for `salePrice`, gated by `reserve2(buyerSlot, sellerSlot)` — the compound authority $s_{\text{buyer}} * s_{\text{seller}}$ — and sequences payment before capability delivery to close the residual `RevVault`-bounce hazard on top of the fuel atomicity the gate already guarantees. A liquidation is structurally the same swap with two substitutions:

- (i) the willing buyer becomes a *keeper* (the liquidator), and
- (ii) the trigger becomes a *health-factor observable* rather than a counterparty’s volition — a `makeGet`-style guard (as in `spj_loans_cost.rho`) fired by the mark crossing the maintenance threshold, not by a block horizon.

The mark-to-market it reads is the futures file’s net-against-`spot`; the seize-and-close is `reserve2` over the position’s collateral purse and the keeper’s payment purse; the incentive (the liquidation bonus) is an ordinary spread on that swap — itself an ordering surface best cleared in a batch (§6).

Finding 3 (Liquidation is assembled, not invented). Liquidation = `makeSellLoan` (loan sale, present) $\circ$ mark-to-market (futures netting, present) $\circ$ a health-guarded `get` (guard combinator, present). It remains a new build — the trigger loop, the keeper incentive, and the batch interaction are new — but it is composed from three primitives already in front of us, and it must sit below oracle finality (Invariant 7.1 of [7]) exactly as option redeem does.

The contrast that sharpens the point. Term lending as implemented settles repayment by *starvation*: a missed scheduled payment is a `schedulePayment` that finds no funded token and simply blocks. That is the *opposite* of liquidation, which must *force* settlement on a solvency breach rather than wait for it. The implemented loans have no collateral purse and no forced-unwind path; liquidation is exactly the machinery they lack (§9).

9 Lending: the instrument is present, the venue is not

We have read `spj_loans_cost.rho` in full. It implements *fixed-schedule, bilateral, term lending*, and it does so correctly.

What is present. Three loan shapes — short-term bullet (`makeShortTermLoan`), mid-term coupon-plus-bullet (`makeMidTermLoan`), long-term flat-amortising (`makeLongTermLoan`). Each decomposes cleanly: disbursement is `give(get(start, truncate(start, one(principal))))` funded by the lender; repayment is `makeScheduledPayments` — a `cand`-folded strip of scheduled payments, each `get(truncate(one(amount)))` — funded by the borrower, each leg its own funded deployment at its own block under concurrent acceptance (the Pattern-D bundle of [5, §4]). Interest is rational (`rateNum/rateDenom`); amounts are `nanoREV`; the registry and `RevVault` plumbing are real. `makeSellLoan` adds a genuine secondary-market primitive: the lender’s repayment position sold atomically via `reserve2`. As term credit — the SPJ zero-coupon-bond / annuity lineage executed with located fuel and atomic settlement — this is complete.

What is structurally absent. This is the credit *instrument*, not the credit *venue*. The gap is not incidental; it is structural along four axes the file never touches:

1. **No pool / no fungible lender side.** Lending is one known lender to one known borrower over a delivery channel. A money market is many suppliers into a shared pool with a fungible supply claim (the `aToken/cToken` receipt). That shared-pool object is the *same* stateful pool of §5 — a money market is an AMM’s sibling, not a loan’s.
2. **No utilisation-driven rate.** Interest is fixed at origination. A money market’s rate is a function of pool utilisation, recomputed continuously — the interest-rate model is the whole point, and it is absent.
3. **No collateral, hence no liquidation.** Every loan is unsecured and calendar-settled; there is no collateral purse, no collateral factor / LTV, no health factor, no mark-to-market, no keeper. Repayment-by-starvation is the opposite of forced liquidation (§8).
4. **No revolving credit.** The schedule is baked at construction: no repay-anytime, borrow-more-anytime, or variable principal.

Finding 4 (Credit instrument vs. credit venue). Term lending is present and correct and lands in the instrument layer as-is. Money-market lending is a distinct sub-venue that *reuses the pool object* (§5) for its supply side and the *liquidation engine* (§8) for its solvency discipline, and adds only a utilisation-curve rate model of its own. The honest framing, exactly parallel to [5] giving the futures instrument while [7] had to build the venue around it: you have the credit instrument but not the credit venue.

10 Structural findings

These are the intellectually load-bearing observations — the parts a formal write-up would develop at length.

10.1 The swap is a theorem, not a construction

Previously the atomic swap was derived from Pattern-A netting. The join schema gives the cleaner derivation: a two-asset swap is a compound-signature join, and conservation of authority (Prop. 4.7 of [3]) says the join gathers exactly one authority per leg or does not fire, *regardless of how tokens are grouped*. Atomic-or-nothing swap is thus a theorem of the schema (Rule 3 / Case J2 / Def. 4.6 of [3]), not an optimisation installed by hand — and the `reserve2`-gated `makeSellLoan` of `spj_loans_cost.rho` is its *already-implemented witness*. Two corollaries:

Proposition 1 (Baskets are free and cannot silently unbundle). Reverse Currying [3, §4.8.4] lets the splitter/joiner assemble and disassemble compound authorities, so index tokens, wrap/unwrap, and basket assets are immediate. *Weakening-forbidden* [3, §4.8.5] is conservation-of-assets by construction: one cannot consume a basket $X * Y$ to discharge a demand for X alone and silently drop Y ; unbundling must be an explicit, observable **Split**. An index product cannot leak its constituents.

Proposition 2 (Coincidence-of-wants is an N -leg join). A ring trade $X \rightarrow Y \rightarrow Z \rightarrow X$ is a compound-signature N -leg join: all legs settle or none, with no merge analysis. The clearing that internalises it is the cross-pool batch of §6.

10.2 The ILL backbone: asset as linear resource, route as curried composition

Remark 4.9 of [3] makes $*$ a tensor on signatures and $\multimap$ its right adjoint, with $(s_1 * s_2) \multimap s_3 \cong s_1 \multimap (s_2 \multimap s_3)$, and leaves open whether the token discipline is a model of ILL with $*$ = $\otimes$ and $\multimap$ the linear function space. For a DEX that open question is the spine:

- an **atomic multi-asset swap** is a $\otimes$ of asset authorities — joint, all-or-nothing (§10.1);
- a **multi-hop route** $X \rightarrow Y \rightarrow Z$ is the curried chain $X \multimap (Y \multimap Z)$ — consume X for the authority to consume Y for Z , staged, exactly the linear-transfer sugar $\multimap$ of [3, §4.9];
- **route atomicity** — the acceptance gate funds the whole route before any hop, so the classic “stranded in the intermediate token” failure is structurally impossible — is precisely the statement that the curried chain discharges as *one* linear proof.

Finding 5 (The ILL reading is the DEX type theory). “An asset is a linear resource, a swap is a linear map, a route is a curried composition, and a well-typed atomic route is a linear proof” is not an analogy but the ILL reading of the token discipline. It is the natural home for the spatial-behavioural types of [5, §8] applied to trading: slippage bounds and pool conservation become type-level assertions (Φ_{sep} for no-aliasing across pools, a graded $\langle \text{swap} \rangle$ modality for bounded-price settlement). Settling the adjunction — is the discipline a model of ILL? — is the theoretical heart of the DEX paper, a sequel the foundational paper already promises.

10.3 Flash swaps: an elegant fit, not a friction

A flash loan is funded by its own proceeds, which naively violates “prove funded before any step fires.” On this substrate it has *two* independent supports. The funding slot [3, §5.7] lets the funder be determined at run time, not compile time: the flash operation creates a fresh slot, and the proceeds fund it mid-execution. And the $\multimap$ staging [3, §4.9] puts the borrow leg (funds the rendezvous) and the repayment leg (funds the continuation) in one atomic scope: the flash operation closes its own loop or does not fire, with no separate reentrancy guard — reentrancy is structurally absent because the unit of state update is a single for-comprehension [3, Rmk.8.2]. **Verdict: flash swaps are a clean fit, plausibly cleaner than the callback-and-check pattern of incumbent chains.**

11 The DEX-launch configuration tuple over a fixed safety floor

Following [7, §5], a launch is the instantiation of a dependent record: some fields’ admissible values depend on earlier fields, and a safety floor is a refinement invariant every instantiation must

satisfy. Permissioning is a field (both spot and derivatives), realising the requested “both, as a configuration” posture.

$$\text{DEX} := \langle \underbrace{\text{pair, assetLines}}_{\text{instrument}}, \underbrace{\text{structure, execution, curve, fees,}}_{\text{microstructure}}, \underbrace{\text{leverage, margin, oracle, spec, dispute, liquidation,}}_{\text{derivatives}}, \underbrace{\text{compliance, limits, gov.}}_{\text{policy}} \rangle$$

Fields (with dependent edges).

- **structure** $\in \{\text{AMM, CLOB, Hybrid}\}$; **execution** $\in \{\text{Continuous, Batch}(w)\}$, orthogonal, but Continuous opts into the contended-price externality [7, §6.2] and Batch is clean by construction. **curve** $\in \{\text{CPMM, LMSR-style, none}\}$ dependent on structure.
- **leverage** present $\Rightarrow$ **margin** $\in \{\text{Isolated, Cross}\}$ and **liquidation**-policy required, and **oracle/spec/dispute** required (derivatives fields are ill-typed absent an oracle, exactly as in [7, §5.2]). Spot-only markets leave the entire derivatives block empty — the oracle-free spine.
- **compliance** $\in \{\text{Open, Gated}(s_{\text{compliance}})\}$: the gated case makes every transfer $\{\text{transfer}\}_{s_{\text{party}} * s_{\text{compliance}}}$ [3, §2.4], the single field that most distinguishes the permissionless from the regulated shape. **limits** are the spatial-type bounds of [5, §8].

The safety floor (fixed across every cell).

1. **Pool conservation with Φ_{sep}** : reserves and purses do not alias across pools; the constant-function invariant never decreases on a swap.
2. **Atomic swap-or-revert**: every swap is a compound-signature join (Prop. 4.7), all-or-nothing.
3. **Route funded before any hop**: a multi-hop route discharges as one linear proof (§10.2); no intermediate-token stranding.
4. **Margin proof before leverage**: no leveraged position opens without a margin funding proof, and the standing gate (Finding 2) governs its life.
5. **Liquidation below oracle finality**: forced unwind is nested below consensus and dispute finality [7, Inv. 7.1], as option redeem is.
6. **Weakening-forbidden**: no silent unbundling of a basket or compound authority (Prop. 1).
7. **Cost is not amount**: value lines and the phlogiston line are separate quantales gated against their product [6, §6], never conflated.

The launcher picks policy above this line; nobody picks out of the floor. That line — configurable policy over fixed safety — is what stops “community selects a market type” from ever meaning “community configures an unsafe market.”

DEX component	F1R3Fi apparatus	Status
Fungible asset line	cost spectrum; minting [3, §2.4]; <code>REV/RevVault</code>	present
Value magnitude	located purse, <code>nanoREV</code> (Finding 1)	present (convention)
Custody / balance	located stacks on unforgeable channels [5, §3.2]	present
Asset-for-asset swap	compound-sig join; <code>reserve2</code> ; <code>Exchange</code> [3, B.9]	present (theorem)
Basket / index token	reverse Currying + weakening-forbidden [3, §4.8]	present
Slippage guard	conservative-bound gate; <code>makeGet</code> -style guard	thin wrapper
Multi-hop route	$\rightarrow$ curried chain [3, §4.9]; one linear proof	thin wrapper
Spot price observable	persistent channel [5, §2.2]	present
TWAP	accumulator on the observable	new (small)
AMM pool (X, Y)	Def. 1; curve from [7, §6.1]	new (core)
LP token / add-remove	share line + pro-rata (§5)	new (small)
Order book	resting funded comprehensions [7, §6.3]	present (idiom)
Order cancellation	object-capability receive on order purse [7, §6.3]	present
Batch clearing	uniform-price join, MEV-flat [7, Prop. 6.1]	present
Cross-pool / CoW clear	N -leg join (Prop. 2); welfare match	new (hard)
Flash swap	funding slot [3, §5.7] + $\rightarrow$ (§10.3)	present (idiom)
Cash-settled future	Pattern-A netting; <code>spj-futures-cost.rho</code>	present
Perp mark / funding	attested observable [7, §4] + futures netting	reused + present
Option settlement	resolution latch [7, §7]	reused
Margin accounting	health observable per position	new (wrapper)
Liquidation engine	standing gate + health-triggered <code>makeSellLoan</code>	new (assembled)
Term loan	<code>makeShortTermLoan</code> etc.; <code>spj_loans_cost.rho</code>	present
Secondary-market sale	<code>makeSellLoan</code> (<code>reserve2</code> swap)	present
Money-market pool	pool object (§5) + supply-claim line	new (reuses pool)
Utilisation rate model	utilisation $\rightarrow$ rate function	new
Compliance gate	$\{\cdot\}_{s_{\text{party}} * s_{\text{compliance}}}$ [3, §2.4]	present
Audit trail	native ledger of funded rendezvous [5, §6]	present
Position limits	Φ_{sep} , spatial bounds [5, §8]	type layer (in progress)

Table 2: Primitive-by-primitive map. The four **bold** rows are the genuinely new builds; the rest are present, reused, or thin wrappers.

12 Primitive-by-primitive map

Table 2 maps each DEX component onto apparatus in the four papers and five implementation files. “present” means realised in an implementation or a prior paper; “reused” means imported from the prediction-market venue unchanged; “new” means to be built, with the build’s nature noted.

The map’s verdict is the note’s headline: of the roughly thirty components a full hybrid DEX needs, four are the real build — the **stateful pool**, **cross-pool batch clearing**, the **liquidation engine**, and the **money-market rate model** — and everything else is present, reused, or a thin wrapper.

13 Phased build plan

A phased plan, each phase the smallest end-to-end slice that exercises new machinery against a test oracle, in the manner of [7, §8.1].

Phase 0 — asset lines.

Fix Finding 1: value as magnitude-in-purse, a second value line beside `REV`, minted per [3, §2.4]. Exercise transfer, custody, and the $\mathcal{Q}/\Phi$ product gate. Smallest slice; no pool.

Phase 1 — spot swap and pool.

Implement Def. 1 for one (X, Y) constant-product pool with LP tokens, add/remove liquidity, and the compound-signature swap (present as a theorem, §10.1). Test oracle: the pool invariant never decreases; Φ_{sep} holds. This is the minimal spot DEX and it is oracle-free.

Phase 2 — batch clearing, hybrid.

Add resting orders (funded comprehensions) and the uniform-price batch join with the curve as residual counterparty [7, §6.2]. Start batch because it is MEV-flat and idiomatic; the hybrid AMM+CLOB is one clearing discipline. Routing ($\multimap$ chains) and slippage guards land here.

Phase 3 — money market.

Reuse the Phase-1 pool for the supply side; add the supply-claim line and the utilisation-rate model. Borrowing draws against pooled liquidity with a collateral purse. Still no forced unwind — defer liquidation to Phase 5.

Phase 4 — derivatives, oracle-priced.

Import the oracle discipline and resolution latch [7, §4, §7]. Perp mark + funding (futures netting), option settlement (latch), cash-settled on a deterministic-stratum feed. Margin accounting as a live health observable.

Phase 5 — liquidation.

Build the standing-gate trigger loop and the health-triggered `makeSellLoan` (Findings 2, 3), nested below oracle finality, with the keeper incentive cleared in the Phase-2 batch. This closes both the money market (Phase 3) and the derivatives spine (Phase 4).

Phase 6 — cross-pool clearing and compliance.

Generalise the batch to internalise cross-pool arbitrage (Prop. 2) — the hard optimisation. Wire the `Gated( $s_{\text{compliance}}$ )` configuration and position limits. Lift the regulated bundle.

14 Established vs. conjectural; open problems

In the spirit of [6, 7], we separate what the design gets from what it hopes for.

Established (given the cost gate and a normalising fragment).

- The multi-asset ledger, custody, transfer, minting, and the atomic asset-for-asset swap are present or theorems (§4, §10.1); the swap’s atomicity is Prop. 4.7 of [3], witnessed by `makeSellLoan`.
- Baskets are free and cannot silently unbundle (Prop. 1); routes discharge as one linear proof (§10.2); flash swaps fit cleanly (§10.3).
- MEV-flat clearing within a batch round is inherited (Prop. 6.1 of [7]).
- Term lending and secondary-market sale are implemented and correct (§9).
- Liquidation is assembled from present primitives (Finding 3), though the trigger loop and incentive are new.

Conjectural / delegated / next.

- **The ILL model is open.** Whether the token discipline is a model of intuitionistic linear logic with $*$ = $\otimes$ and $\multimap$ the linear function space is the theoretical heart (Finding 5) and is unsettled in [3]. The DEX types (slippage, conservation) sit on top of that answer.
- **Cross-pool welfare clearing is mechanism design.** Internalising arbitrage into a multi-asset batch is an optimisation problem (Prop.2), the cousin of the #P-hard combinatorial-pricing caveat [7, §3]; which objective (surplus vs. coverage) and which matching algorithm is policy, not a theorem, exactly as the clearing rule is in [6, §12].
- **The money-market rate model is design.** A utilisation-curve rate, its incentive compatibility, and its interaction with liquidation are engineering choices with no calculus-forced answer.
- **Oracle risk is inherited, not solved.** Everything mark-priced inherits the prediction-market oracle trust model and its open dispute-resolution question [7, §4,§8.5], now with leverage at stake.
- **Liveness is a market property.** That a pool always has liquidity, or a liquidation always finds a keeper, depends on incentives and participation; escrow and located purses give safety, not liveness — the same separation [6, §12] draws.

Suggested next decisions. (1) Fix the value-line convention (Finding 1) and build Phase 0–1. (2) Settle, or bound, the ILL question enough to state the DEX type discipline. (3) Choose the batch clearing objective and matching algorithm. (4) Specify the utilisation-rate model. (5) Decide the liquidation keeper incentive and its batch interaction.

15 Conclusion

A financial contract is a thing one holds; a prediction market is a place one trades; a DEX is a stack of such places sharing a collateral and settlement layer. The combinators, realised in the cost-accounted rho calculus, already give the instruments. We have shown that the venues are mostly within reach of the same apparatus, and that the reach is cleanly organised by a two-spine decomposition: the spot spine settles on-substrate and oracle-free, inheriting atomic swap (a theorem of the join schema), MEV-flat batch clearing, and located custody; the derivatives spine re-imports the prediction-market oracle discipline wholesale and forces exactly one new risk-centre object, liquidation, which is the acceptance gate lifted to a standing margin invariant and, operationally, a health-triggered variant of the loan sale already implemented. The multi-asset ledger falls out of the cost spectrum; the tensor–hom adjunction is the venue’s intuitionistic-linear-logic backbone, under which an asset is a linear resource, a swap a linear map, and an atomic route a single linear proof. Of the roughly thirty components a full hybrid venue needs, four are the genuine build — the stateful pool, cross-pool batch clearing, the liquidation engine, and the money-market rate model — and the credit venue, as distinct from the credit instrument, is the one place where present machinery names a gap it does not yet fill. A description language tells you what a contract is worth; a settlement language on this substrate tells you, before a token moves, that it will settle; and an exchange on this substrate tells you, before a market opens, the exact discipline under which every asset in it will trade, lever, and unwind.

Acknowledgements. This note develops work conducted in extended dialogue with Anthropic’s Claude, which served as a sounding board throughout — decomposing the venue into spines, tracing the asset layer back to the cost spectrum, reading the loan and futures implementations to separate the credit instrument from the credit venue, and pressing the liquidation-as-standing-gate and swap-as-theorem identifications. The ideas and any errors are the author’s.

References

- [1] L. G. Meredith, M. Radestock. A reflective higher-order calculus. *ENTCS* 141(5):49–67, 2005.
- [2] L. G. Meredith, M. Radestock. Namespace logic: a logic for a reflective higher-order calculus. *TGC*, LNCS 3705:353–369, Springer, 2005.
- [3] L. G. Meredith. Cost-Accounted Rho Calculus: A Spectral Decomposition of Phlogiston. F1R3FLY.io, 2026.
- [4] L. G. Meredith. Continued interactive GSLTs and the cost endofunctor: a construction one level up from cost-accounted Rholang. F1R3FLY.io, 2026.
- [5] L. G. Meredith. Financial Contract Combinators in the Cost-Accounted Rho Calculus. F1R3FLY.io, 2026.
- [6] F1R3FLY.io (in dialogue with D. Spivak, Topos Institute). Priced Plausible Fiction: A Two-Sided Market of Programmatic Intentions on the Cost-Accounted Rho Calculus. 2026.
- [7] L. G. Meredith. Prediction Markets on the Cost-Accounted Rho Calculus. F1R3FLY.io, 2026.
- [8] L. G. Meredith. MeTTaIL (Rholang 1.4): a language for defining languages. F1R3FLY.io, 2025–2026.
- [9] L. G. Meredith et al. Rholang Specification. F1R3FLY.io / RChain Cooperative, 2017–2026.
- [10] S. Peyton Jones, J.-M. Eber, J. Seward. Composing contracts: an adventure in financial engineering (functional pearl). *ICFP*, Montreal, 2000.
- [11] F1R3FLY.io. F1R3Fi cost-accounted instrument implementations: `spj_contracts_cost.rho`, `spj_dutch_auction_cost.rho`, `spj_loans_cost.rho`, `spj_futures_cost.rho`, `spj_power_contracts_cost.rho`. Open-source repository, 2026.
- [12] H. Adams et al. Uniswap v2 Core / constant-product automated market makers. 2020.
- [13] R. Hanson. Logarithmic market scoring rules for modular combinatorial information aggregation. *Journal of Prediction Markets* 1(1):3–15, 2007.
- [14] Money-market protocols with utilisation-curve interest and collateralised liquidation (Compound, Aave). Protocol documentation, 2019–2023.
- [15] Batch-auction / coincidence-of-wants DEX designs (CoW Protocol, uniform-price batch clearing). 2021–2024.
- [16] P. Daian et al. Flash Boys 2.0: frontrunning, transaction reordering, and consensus instability in decentralized exchanges. *IEEE S&P*, 2020.

- [17] M. S. Miller. Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control. PhD thesis, Johns Hopkins University, 2006.
- [18] J.-Y. Girard. Linear logic. *Theoretical Computer Science* 50(1):1–101, 1987.
- [19] V. Buterin, V. Griffith. Casper the Friendly Finality Gadget. arXiv:1710.09437, 2017.
- [20] Gnosis. Conditional Tokens Framework. 2019.